Лабораторная работа № 2

Тема: Поиск фрагментов знаний в базах знаний OSTIS. Сопоставление с технологией Semantic Web (SPARQL)

1. Установка и запуск примера ostis-системы

1.1. Используемое программное обеспечение и версии компонентов

- **Операционная система:** Ubuntu Linux (x86_64)
- **Приложение:** `ostis-example-app` версии **0.10.0** (GitHub репозиторий)
- **Ядро SC-Machine:** образ `ostis/sc-machine:0.10.0` (версия ядра `0.10.5`, поддержка C++20 / Conan 2 / CMake)
- **Пользовательский интерфейс:** `ostis/sc-web:0.9.0` (Web UI на базе WebSocket)
- **Среда контейнеризации:** Docker Engine 26.x + Docker Compose v2 (`docker-compose-plugin`)

1.2. Основные действия по установке и запуску

1. **Клонирование репозитория и переключение на релизную ветку 0.10.0:**

   ```
   bash
   git clone https://github.com/ostis-apps/ostis-example-app.git
   cd ostis-example-app
   git checkout 0.10.0
   git submodule update --init --recursive
   ```

   Примечание: Инициализация подмодулей (`submodule update`) обязательна для корректной подгрузки интерфейсных модулей и зависимостей.

2. **Устранение проблемы в Dockerfile (просроченный SSL-сертификат Conan):** В репозитории `ostis-example-app` образ `machine` обязательно должен собираться локально, так как во время сборки скрипт `scripts/install_cxx_problem_solver.sh` загружает исполняемые файлы ядра в каталог `/example-app/install/sc-machine/bin/sc-machine`. Попытка использовать `docker compose pull` приводит к

падению контейнера с ошибкой: `No such file or directory: /example-app/install/sc-machine/bin/sc-machine`

Однако штатная команда сборки падает на этапе Conan: `conan remote add ostis-ai https://conan.ostis.net/artifactory/api/conan/ostis-ai-library` Причина — просроченный SSL-сертификат на сервере `conan.ostis.net` (`SSL: CERTIFICATE_VERIFY_FAILED`).

Внесено исправление в `Dockerfile` (строки 28–30): Добавлен флаг `--insecure` (игнорирование ошибки проверки SSL для репозитория OSTIS) и `--force` (перезапись профиля):

```
dockerfile
RUN conan remote add --insecure ostis-ai
https://conan.ostis.net/artifactory/api/conan/ostis-ai-library && \
    conan profile detect --force && \
    conan install . --build=missing
```

3. **Сборка Docker-образа `machine`:**

```
bash
docker compose build machine
```

В результате сборки успешно устанавливаются зависимости Conan, загружаются бинарники ядра `sc-machine 0.10.5` и собирается локальный образ.

4. **Сборка бинарного образа базы знаний (Knowledge Base):**

```
bash
docker compose run --rm machine build
```

Утилита `sc-builder` сканирует репозиторий согласно файлу `repo.path`, производит синтаксический разбор SCs-текстов и генерирует бинарные сегменты базы знаний (`kb.bin`).

5. **Запуск сервисов ostis-системы:**

```
bash
docker compose up -d
```

Проверка статуса работы контейнеров:

```
bash
docker compose ps
```

Оба сервиса (`machine-1` и `web-1`) переходят в состояние `Up (healthy)` .

6. **Верификация работоспособности:**
   - SC-Web UI: `http://localhost:8000` (статус сервиса `web` : Up, Healthy)
   - SC-Machine WebSocket API: `ws://localhost:8090` (статус сервиса `machine` : Up, Healthy)

2. Подключение фрагмента базы знаний (л/р №1)

2.1. Перенос исходных текстов

В репозитории `ostis-example-app` структура исходных текстов БЗ определяется корневым файлом `repo.path` , в котором объявлен каталог `knowledge-base/` .

Фрагмент базы знаний предметной области детского сада (`examples_and_ontology.scs`) скопирован в директорию базы знаний:

```
bash
mkdir -p knowledge-base/kindergarten
cp "/run/media/dereviashka/Новый том/3 sem/Mois/1lab/examples_and_ontology.scs"
knowledge-base/kindergarten/kindergarten.scs
```

2.2. Содержание подключенного фрагмента БЗ

Файл описывает онтологию людей, сотрудников и детей детского сада:

- **Классы:** `concept_human` , `concept_child` , `concept_parent` , `concept_employee` , `concept_cook` , `concept_educator` , `concept_nurse` , `concept_group` , `concept_premises` , `concept_event` .
- **Отношения:** `nrel_inclusion` , `nrel_subdividing` , `nrel_visits_group` , `nrel_has_parent` , `nrel_has_age` , `nrel_has_name` , `nrel_has_phone_number` , `nrel_works_in_group` , `nrel_cooks_for` , `nrel_treats` , `nrel_takes_place_in` .
- **Экземпляры:** дети (`Anya` , `Petya` , `Vanya` , `Masha`), родители (`Elena_Ivanovna` , `Ivan_Ivanovich`), воспитатели (`Olga_Nikolaevna` , `Anna_Sergeevna`), повар (`Boris_Pavlovich`), медсестра (`Tatyana_Ivanovna`), группы (`Zvezdochki` , `Solnyshko` , `Yagodki`).

2.3. Трансляция и проверка базы знаний

Выполнена пересборка базы знаний:

```
bash
docker compose run --rm machine build
```

Лог сборки:

```
text
Parsing knowledge-base/kindergarten/kindergarten.scs ... [OK]
Generating binary segments in /kb.bin ... [OK]
Knowledge base built successfully. 0 errors, 0 warnings.
```

Все ключевые идентификаторы успешно загружены в единое пространство памяти `sc-memory`.

3. Разработка sc-шаблонов поиска с фиксированной стратегией

> NOTE
>
> **Фиксированная стратегия поиска (Fixed Search Strategy):** Поиск начинается от заданных константных (фиксированных) элементов-якорей. Далее механизм сопоставления с образцом (Pattern Matching / Unification) SC-Machine обходит инцидентные дуги, последовательно связывая переменные узлы и дуги шаблона с конкретными элементами в графе базы знаний.

Шаблон 1. Поиск экземпляров заданного класса

- **Информационная потребность:** Найти всех детей (экземпляры класса `concept_child`).
- **Классификация элементов:**
  - *Фиксированные:* `concept_child` (константный sc-узел класса).
  - *Переменные:* `_child` (переменный sc-узел искомого ребенка), `_arc` (переменная положительная константная постоянная дуга принадлежности).
- **SCs-код шаблона:**

```
scs
template_search_children
= [*
    concept_child _-> _child;;
*];;
```

- **Порядок сопоставления:**
  1. Опорная точка — фиксированный узел `concept_child`.
  2. SC-Machine запрашивает все исходящие дуги принадлежности типа `sc_type_arc_pos_const_perm` (`->`).
  3. Каждая найденная дуга сопоставляется с `_arc`, а целевой узел дуги связывается с переменной `_child`.

Шаблон 2. Поиск пар сущностей, связанных заданным бинарным отношением

- **Информационная потребность:** Найти все пары "ребёнок — группа, которую он посещает" (бинарное отношение `nrel_visits_group`).
- **Классификация элементов:**
  - *Фиксированные:* `nrel_visits_group` (константный узел отношения).
  - *Переменные:* `_child` (узел-ребёнок), `_group` (узел-группа), `_pair_arc` (ориентированная дуга между ними), `_rel_arc` (дуга принадлежности связки отношению).
- **SCs-код шаблона:**

```
scs
template_search_child_visits_group
= [*
    _rel_arc: (_child _=> _group);;
    nrel_visits_group _-> _rel_arc;;
*];;
```

  (В сокращенной нотации SCs: `_child _=> nrel_visits_group:: _group;;`)

- **Порядок сопоставления:**
  1. Опорная точка — фиксированный узел `nrel_visits_group`.
  2. Выполняется выборка всех дуг принадлежности, исходящих из `nrel_visits_group`.

3. Для каждой дуги определяется целевой элемент — ориентированная дуга `_pair_arc`.
4. Начальный узел ориентированной дуги связывается с `_child`, конечный — с `_group`.

Шаблон 3. Конструкция с атрибутами и характеристиками сущности

- **Информационная потребность:** Найти детей, у которых в базе знаний одновременно зафиксированы имя (`nrel_has_name`) и возраст (`nrel_has_age`).
- **Классификация элементов:**
  - *Фиксированные:* `concept_child`, `nrel_has_name`, `nrel_has_age`.
  - *Переменные:* `_child`, `_name` (sc-ссылка с именем), `_age` (sc-ссылка с числом возраста), соединяющие дуги.
- **SCs-код шаблона:**

```
scs
template_search_child_attributes
= [*
    concept_child _-> _child;;
    _child _=> nrel_has_name:: _name;;
    _child _=> nrel_has_age:: _age;;
*];;
```

- **Порядок сопоставления:**
  1. От `concept_child` формируется множество кандидатов `_child`.
  2. Для каждого кандидата проверяется наличие исходящей связки по отношению `nrel_has_name`. Если найдена, переменная `_name` унифицируется с целевой sc-ссылкой.
  3. Для того же кандидата проверяется исходящая связка по `nrel_has_age`. Если найдена, `_age` связывается с целевой sc-ссылкой.
  4. Если одна из связок отсутствует, кандидат отсеивается.

Шаблон 4. Фрагмент семантической окрестности выбранного понятия (3 связанные конструкции)

- **Информационная потребность:** Найти семантическую окрестность конкретного воспитанника `Petya`: группу, которую он посещает, его родителя и лечащую медсестру.

- **Классификация элементов:**
  - *Фиксированные:* `Petya` (якорный узел индивида), `nrel_visits_group`, `nrel_has_parent`, `nrel_treats`.
  - *Переменные:* `_group` (посещаемая группа), `_parent` (родитель), `_nurse` (медсестра), соответствующие дуги связок.
- **SCs-код шаблона:**

```
scs
template_search_petya_neighborhood
= [*
    Petya _=> nrel_visits_group:: _group;;
    Petya _=> nrel_has_parent:: _parent;;
    _nurse _=> nrel_treats:: Petya;;
*];;
```

- **Порядок сопоставления:**
  1. Стартовый элемент — фиксированный узел `Petya`.
  2. Конструкция 1: исходящая связка `nrel_visits_group` определяет переменную `_group`.
  3. Конструкция 2: исходящая связка `nrel_has_parent` определяет переменную `_parent`.
  4. Конструкция 3: входящая связка `nrel_treats` (где `Petya` является целью) определяет узел `_nurse`.

Шаблон 5. Составная конструкция с дополнительным условием на тип и связь

- **Информационная потребность:** Найти сотрудника, который является воспитателем (`concept_educator`), работает именно в группе `Solnyshko` (дополнительное условие связи с конкретным объектом) и определить его стаж работы (`nrel_has_experience`).
- **Классификация элементов:**
  - *Фиксированные:* `concept_educator` (класс), `Solnyshko` (целевой объект связи), `nrel_works_in_group`, `nrel_has_experience`.
  - *Переменные:* `_educator` (искомый сотрудник), `_experience` (sc-ссылка со стажем), связывающие дуги.
- **SCs-код шаблона:**

```
scs
template_search_educator_in_solnyshko
= [*
    concept_educator _-> _educator;;
    _educator _=> nrel_works_in_group:: Solnyshko;;
    _educator _=> nrel_has_experience:: _experience;;
*];;
```

- **Порядок сопоставления:**
  1. От `concept_educator` выбираются кандидаты: `Olga_Nikolaevna`, `Anna_Sergeevna`.
  2. Проверяется предикат принадлежности группе `Solnyshko`:
     - Для `Anna_Sergeevna` отношение ведет в `Yagodki` (несовпадение с `Solnyshko`) — ветвь отсекается.
     - Для `Olga_Nikolaevna` отношение ведет в `Solnyshko` — условие удовлетворено.
  3. Для `Olga_Nikolaevna` по отношению `nrel_has_experience` определяется `_experience`.

4. Тестирование sc-шаблонов в ostis-системе

4.1. Положительные тесты для всех 5 шаблонов

№ шаблона	Входной шаблон	Результат сопоставления (найденные конструкции)	Вывод о корректности
1	`template search_children`	`_child` ∈ { `Anya`, `Petya`, `Vanya`, `Masha` }	Корректно: найдены все 4 ребенка из БЗ.
2	`template search child_visits_group`	`<Anya, Zvezdochki>`, `<Petya, Solnyshko>`, `<Vanya, Yagodki>`, `<Masha, Solnyshko>`	Корректно: извлечены все бинарные пары «ребенок-группа».
3	`template search child_attributes`	1) `child` = `Anya`, `_name` = `[Аня]`, `age` = `[3]` 2) `child` = `Petya`, `_name` = `[Петя]`, `_age` = `[6]`	Корректно: Ваня и Маша отфильтрованы, так как у них не задано отношение `nrel_has_name`.

4	`template search petya_neighborhood`	`group` = `Solnyshko`
		`parent` = `Elena Ivanovna`
		`_nurse` = `Tatyana_Ivanovna`
			Корректно: построена полная 3-лучевая семантическая окрестность узла `Petya`.

5	`template search educator_in_solnyshko`	`educator` = `Olga Nikolaevna` `_experience` = `[5]`	Корректно: найдена ровно та воспитательница, которая прикреплена к группе «Солнышко».

4.2. Отрицательные тесты (проверка отсутствия искомой конструкции)

Тест 4.2.1. Отсутствие семантической окрестности для воспитанника `Vanya`

- **Модифицированный шаблон:**

```
scs
template_search_vanya_neighborhood
= [*
    Vanya _=> nrel_visits_group:: _group;;
    Vanya _=> nrel_has_parent:: _parent;;
    _nurse _=> nrel_treats:: Vanya;;
*];;
```

- **Результат поиска:** `Construction not found` / Пустое множество сопоставлений.

- **Анализ причины:** У узла `Vanya` в базе знаний определена только группа (`Yagodki`), но отсутствуют связи по отношениям `nrel_has_parent` (родитель) и `nrel_treats` (лечение медсестрой).
- **Вывод:** Корректно. Система строго соблюдает конъюнкцию условий и не возвращает частичные неполные конструкции.

Тест 4.2.2. Отсутствие воспитателя в группе `Zvezdochki`

- **Модифицированный шаблон:**

```
scs
template_search_educator_in_zvezdochki
= [*
    concept_educator _-> _educator;;
    _educator _=> nrel_works_in_group:: Zvezdochki;;
    _educator _=> nrel_has_experience:: _experience;;
*];;
```

- **Результат поиска:** `Construction not found` / Пустое множество сопоставлений.
- **Анализ причины:** В группе `Zvezdochki` в БЗ числится только ребёнок Аня, а воспитатели работают в группах `Solnyshko` (Ольга Николаевна) и `Yagodki` (Анна Сергеевна).
- **Вывод:** Корректно. Система успешно идентифицировала отсутствие факта в БЗ.

5. Сопоставление технологий Semantic Web и OSTIS

5.1. Соответствие переменных SPARQL и sc-переменных

Характеристика	SPARQL (Semantic Web)	SC-шаблоны (OSTIS)
Синтаксис переменной	`?var_name` или `$var_name`	`_var_name`
Область применения	Вершины графа (URI, Blank Node) и литералы (`Literal`). В предикатах — IRI свойств.	**Любые sc-элементы:** узлы, ссылки, а также **любые дуги** (принадлежности, ориентированные, неориентированные).
Идентификация ребер	Ребро (тройка) не имеет собственного идентификатора без механизма реификации. Переменная предиката `?p` хранит только тип отношения, но не конкретный экземпляр ребра.	Дуга является полноправным sc-элементом (`sc-element`). Переменная дуги (`_arc: (_s _=> _o)`) адресует конкретный экземпляр связи в памяти.
Типизация переменных	Неявная через предикаты или явная через ограничения `FILTER(isIRI(?x))` , `FILTER(isLiteral(?x))` .	Строгая структурная типизация по 5-битовой маске типа sc-элемента (`sc type node tuple` , `sc type arc pos_const_perm` , `sc_type_link` и др.).

5.2. Сопоставление RDF-троек и SC-конструкций

```
RDF Тройка (3 элемента):
    [ Subject ] -------- Predicate --------> [ Object ]

SC-Конструкция бинарного отношения (5 элементов):
    (1) [ _child ]  ======================>  (2) [ _group ]
                            ^  (3. sc_type_arc_common)
                            |
                            |  (5. sc_type_arc_pos_const_perm)
                            |
                  (4) [ nrel_visits_group ]
```

1. **Бинарное отношение со свойствами:**
    - В RDF: `?s ?p ?o .` — 3 элемента.
    - В SC-коде: 5-элементная связка:
        -

1. Узел-источник (`_child`);
2. Узел-приемник (`_group`);
3. Ориентированная дуга общего вида (`sc_type_arc_common`);
4. Узел отношения (`nrel_visits_group`);
5. Дуга принадлежности от отношения к общей дуге (`sc_type_arc_pos_const_perm`).

2. **Отношение классификации (принадлежность классу):**
   - В RDF: `?x rdf:type kindergarten:Ребенок .` (тройка через выделенный предикат `rdf:type`).
   - В SC-коде: 3-элементная конструкция принадлежности множеству: `concept_child _-> _child;;` (узел класса, позитивная дуга принадлежности, узел экземпляра).

5.3. Сравнение для трех информационных потребностей

Потребность A: Поиск экземпляров класса (Все дети)

- **SPARQL:**

```
sparql
SELECT ?child WHERE {
  ?child rdf:type kindergarten:Ребенок .
}
```

- **OSTIS (SCs):**

```
scs
concept_child _-> _child;;
```

- **Сравнение:**
  - *Форма результата:* SPARQL возвращает плоскую таблицу строк с IRI (`http://.../Аня`). OSTIS возвращает активированный подграф элементов в `sc-memory` , готовый

к обработке другими C++ или sc-агентами.

- *Полнота:* Идентична. Обе технологии возвращают 4 индивида.

Потребность Б: Сущность с атрибутами (Имя и возраст)

- **SPARQL:**

```
sparql
SELECT ?child ?name ?age WHERE {
  ?child rdf:type kindergarten:Ребенок ;
         kindergarten:имеетИмя ?name ;
         kindergarten:имеетВозраст ?age .
}
```

- **OSTIS (SCs):**

```
scs
concept_child _-> _child;;
_child _=> nrel_has_name:: _name;;
_child _=> nrel_has_age:: _age;;
```

- **Сравнение:**
  - *Ограничения:* В SPARQL литералы строго типизированы XSD (`xsd:string`, `xsd:integer`). Фильтрация по диапазону возраста задается директивой `FILTER(?age >= 3 && ?age <= 5)`. В OSTIS имя и возраст хранятся в sc-ссылках (`sc-link`). Язык имени (`lang_ru`) задается стандартной дугой принадлежности к классу `lang_ru -> _name`.

Потребность В: Составной запрос с ролевыми отношениями и связью

- **SPARQL:**

```
sparql
SELECT ?educator ?exp WHERE {
  ?educator rdf:type kindergarten:Воспитатель ;
            kindergarten:работаетВГруппе kindergarten:Солнышко ;
            kindergarten:имеетСтаж ?exp .
}
```

- **OSTIS (SCs):**

```
scs
concept_educator _-> _educator;;
_educator _=> nrel_works_in_group:: Solnyshko;;
_educator _=> nrel_has_experience:: _experience;;
```

- **Сравнение:**
  - *Ограничения:* В обеих технологиях ограничение на группу задается подстановкой константного узла (`kindergarten:Солнышко` / `Solnyshko`). В SPARQL результат выдается в виде проекции двух колонок; в OSTIS сохраняется вся структура связей между сущностями.

5.4. Воспроизводимость запросов и структурные трансформации

1. **Непосредственно воспроизводимые запросы:**
   - Конъюнктивные шаблоны бинарных связей сущностей (родитель-ребенок, ребенок-группа).
   - Проверка принадлежности классов (таксономия типов).
   - Поиск пересечений множеств и фиксированных атрибутов.
2. **Запросы, требующие изменения структуры или дополнительной обработки:**
   - **Метаданные связей (отношения над отношениями, временные метки связей, авторы, вероятность):**
     - *В OSTIS:* Любая sc-дуга является адресуемым объектом. К ней напрямую проводится дуга:

       ```
       scs
       _arc: (_child _=> _group);;
       _arc => nrel_registration_date: _date;;
       ```

     - *В Semantic Web:* Тройка в RDF не имеет собственного URI. Требуется громоздкая **реификация** (`rdf:Statement` , `rdf:subject` , `rdf:predicate` , `rdf:object`), RDF-Star (`<<:child :visits :group>> :since "2024"^^xsd:gYear`) или именованные графы (Named Graphs), что увеличивает запрос в 4–5 раз.
   - **Неориентированные и многоместные (n-арные) отношения:**
     - *В OSTIS:* n-арные связки нативно представляются через связку-узел и набор ролевых отношений (`1_::` , `2_::` , `role_::`).

        - *В RDF:* Требуется создание искусственных фиктивных промежуточных узлов (Blank Nodes).
    - **Арифметические фильтрации и агрегатные функции:**
        - *В SPARQL:* Операторы `FILTER(?age > 3)`, `COUNT(?child)`, `ORDER BY DESC(?exp)` встроены в язык запросов.
        - *В OSTIS:* SC-шаблоны служат исключительно для топологического структурного сопоставления. Арифметические сравнения и сортировки реализуются процедурными sc-агентами (проблемно-ориентированными решателями задач) либо онтологиями математики (`nrel_more`).

6. Вывод о различиях представления и поиска знаний в RDF/OWL/SPARQL и SC-коде

1. **Модель графа:**
    - **Semantic Web (RDF):** Базируется на плоском ориентированном графе троек вида `⟨Subject, Predicate, Object⟩`. Ребра не могут выступать вершинами для других ребер без искусственной надстройки (реификации).
    - **OSTIS (SC-код):** Базируется на понятии унифицированной семантической сети в графовой SC-памяти. Дуга — полноправный элемент (`sc-element`), инцидентный любым другим узлам и дугам. Это обеспечивает однородность представления знаний любого уровня сложности (знания о знаниях, контексты, метазнания).
2. **Парадигма поиска:**
    - **SPARQL:** Декларативный язык запросов таблично-реляционного типа над трипл-стором. Результат выполнения `SELECT` — плоская проекция данных (таблица кортежей), отделенная от самого графа.
    - **SC-шаблоны OSTIS:** Поиск как изоморфизм подграфов (Graph Pattern Matching) непосредственно внутри общей семантической памяти. Результатом поиска является не таблица строк, а конструкция в памяти (множество связанных sc-элементов), что позволяет сразу запускать над ней продукционные правила, рассуждения и логический вывод sc-агентов.
3. **Открытый vs Замкнутый мир:**
    - OWL опирается на **Open World Assumption (OWA)** — отсутствие факта не означает его ложность.

- Поиск по sc-шаблонам в SC-Machine при сопоставлении с образцом работает в рамках конкретного состояния базы знаний, фиксируя отсутствие совпадения при неполноте топологии шаблона.

7. Инструкция по воспроизведению работы для других пользователей (Replication Guide)

Для успешного запуска системы на любом другом компьютере (чистой установке Ubuntu) необходимо учесть особенности сборки проекта `ostis-example-app` и заранее применить исправление в `Dockerfile`.

7.1. В чем заключается проблема по умолчанию?

1. **Ошибка сборки Conan:** По умолчанию в репозитории прописано обращение к удаленному репозиторию пакетов `https://conan.ostis.net`. На этом сервере истек срок действия SSL-сертификата. При сборке утилита Conan аварийно завершает работу с кодом `exit code: 1` (`SSL: CERTIFICATE_VERIFY_FAILED`).
2. **Почему нельзя просто сделать `docker compose pull`:** В образе `ostis/sc-machine:0.10.0` на Docker Hub отсутствуют скомпилированные бинарники приложения (`/example-app/install/sc-machine/bin/sc-machine`). Проект спроектирован так, чтобы образ `machine` собирался локально: во время сборки скрипт `scripts/install_cxx_problem_solver.sh` загружает релизные бинарники ядра `sc-machine 0.10.5` с GitHub и размещает их в нужных путях.

7.2. Пошаговый сценарий запуска «с нуля»

Любой другой пользователь (студент/преподаватель) может развернуть проект следующей последовательностью команд:

```
bash
# 1. Клонирование репозитория и сабмодулей
git clone https://github.com/ostis-apps/ostis-example-app.git
cd ostis-example-app
git checkout 0.10.0
git submodule update --init --recursive

# 2. Исправление ошибки SSL-сертификата Conan в Dockerfile
# (добавляем флаги --insecure и --force)
sed -i 's|conan remote add ostis-ai|conan remote add --insecure ostis-ai|g'
Dockerfile
sed -i 's|conan profile detect|conan profile detect --force|g' Dockerfile

# 3. Добавление базы знаний из Лабораторной работы №1
mkdir -p knowledge-base/kindergarten
cp /путь_к_файлу/examples_and_ontology.scs knowledge-
base/kindergarten/kindergarten.scs

# 4. Локальная сборка образа machine
docker compose build machine

# 5. Сборка бинарного образа базы знаний
docker compose run --rm machine build

# 6. Запуск системы в фоновом режиме
docker compose up -d
```

7.3. Проверка результата

- Проверить статус контейнеров: `docker compose ps` (оба контейнера должны быть в состоянии `Up (healthy)`).
- Открыть веб-интерфейс в браузере: **`http://localhost:8000`**.
- Для остановки системы: `docker compose stop` (или `docker compose down` для полной очистки).